

# GOVERNMENT SCHEME MANAGEMENT SYSTEM

Database Management Systems Project



**Submitted by:**

Kavya Singal (1024030470)  
Ipshita Singla (1024030473)  
Akshaj Singhmar (1024030493)

Under the guidance of:

**Dr. Reaya Grewal**

# TABLE OF CONTENTS

| Chapter No. | Topic                               | Page No. |
|-------------|-------------------------------------|----------|
| 1           | Introduction                        | 1        |
| 1.1         | Problem Statement                   | 1        |
| 1.2         | Key Objectives                      | 1        |
| 1.3         | Requirement Analysis                | 2        |
| 2           | System Overview                     | 3        |
| 3           | ER Diagram                          | 4        |
| 4           | ER to Tables                        | 5        |
| 5           | Normalisation                       | 9        |
| 6           | SQL Code — Table Definitions & Data | 10       |
| 7           | SQL Code — Queries (30 Queries)     | 17       |
| 8           | PL/SQL Code                         | 23       |
| 8.1         | Functions                           | 23       |
| 8.2         | Stored Procedures                   | 26       |
| 8.3         | Triggers                            | 31       |
| 9           | Conclusion                          | 33       |
| 10          | References                          | 34       |

# 1. INTRODUCTION

## 1.1 Problem Statement

India operates thousands of government welfare schemes targeting farmers, rural households, women, SC/ST communities, and persons with disabilities. However, the administration of these schemes suffers from significant inefficiencies: duplicate beneficiaries, ineligible applications slipping through manual checks, delayed fund disbursements, and a complete lack of real-time visibility into budget utilisation.

To address these challenges, the Government Scheme Management System (GSMS) provides a centralised, SQL-powered backend that manages the full lifecycle of scheme administration — from citizen registration and eligibility checking to application approval, Direct Benefit Transfer (DBT) simulation, and audit logging. The system is built entirely on Oracle SQL and PL/SQL, and is designed to run on the Oracle Live SQL platform.

## 1.2 Key Objectives

- Maintain a master registry of government departments and their welfare schemes with budget pools.
- Store citizen profiles with demographic, income, and location data needed for eligibility evaluation.
- Automate eligibility checks using a configurable rule table, eliminating hard-coded logic.
- Manage the application workflow from SUBMITTED through UNDER\_REVIEW, APPROVED, and DISBURSED.
- Simulate Direct Benefit Transfer (DBT) with fund disbursement records tied to bank accounts.
- Track officer performance, processing times, and approval rates for accountability.
- Maintain a tamper-proof audit log of every application status change via triggers.

## 1.3 Requirement Analysis

### 1. Scheme & Department Management:

- Departments must be stored with ministry, department code, and head details.
- Each scheme must have configurable base and maximum benefit amounts, applicable states, and active/inactive status.

### 2. Eligibility Rule Engine:

- Rules must be stored in a table (not hard-coded), covering age range, income range, allowed categories, location type, gender restriction, and minimum land holding.

### 3. Citizen Registry:

- Citizens must be stored with Aadhaar, demographic details, income, occupation, land holding, bank account, and verification status.
- The system must support bulk citizen data generation through a stored procedure for testing purposes.

### 4. Application Workflow:

- An application submission procedure must check duplicates and eligibility before inserting.
- An approval procedure must validate officer authority, check fund availability, compute benefit, disburse funds, and update the fund pool atomically.

### 5. Reporting and Analytics:

- SQL queries must cover basic validation, business logic, analytics, and fraud detection.
- A cursor-based scheme performance report must be printable via DBMS\_OUTPUT.

## 2. SYSTEM OVERVIEW

The GSMS database is structured around three primary domains: Master Data (departments, schemes, eligibility rules, fund pools, officers), Beneficiary Data (citizens, documents), and Transactional Data (applications, disbursements, audit logs). The diagram below illustrates how these domains interact.

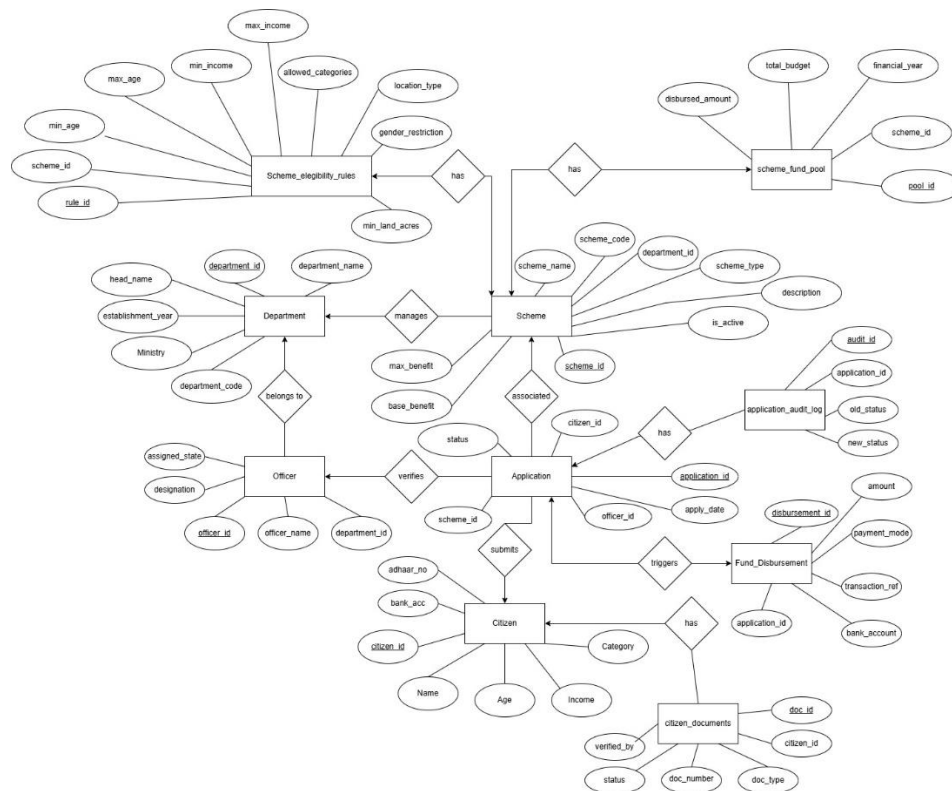
Key design decisions:

- Eligibility rules are externalised into a Scheme\_Eligibility\_Rules table. This means adding a new rule for a scheme requires an INSERT, not a code change.
- The benefit amount is computed dynamically by the `calculate_benefit()` function based on income tier, category, location, and age — so the same scheme pays different amounts to different beneficiaries.
- A priority scoring function (`get_priority_score`) assigns each citizen a score out of 100, allowing officers to process the most vulnerable applicants first.
- Fund pools are maintained in real time: every disbursement triggers an automatic update to Scheme\_Fund\_Pool via a database trigger.
- The application status lifecycle is enforced by constraints and the `approve_application` procedure, preventing illegal transitions.

Technologies used:

- Oracle Database (Oracle Live SQL) for all DDL, DML, and PL/SQL.
- DBMS\_RANDOM for realistic synthetic citizen data generation.
- DBMS\_OUTPUT for procedure output and scheme performance reporting.

## 3. ER DIAGRAM



The entity-relationship model for the Government Scheme Management System consists of the following key entities and relationships:

Entities and their primary attributes:

- Department — department\_id (PK), department\_name, department\_code, ministry, head\_name, established\_year
- Scheme — scheme\_id (PK), scheme\_name, scheme\_code, department\_id (FK), scheme\_type, description, launch\_date, base\_benefit\_amount, max\_benefit\_amount, is\_active
- Scheme\_Eligibility\_Rules — rule\_id (PK), scheme\_id (FK), min\_age, max\_age, max\_income, min\_income, allowed\_categories, location\_type, gender\_restriction, min\_land\_acres
- Scheme\_Fund\_Pool — pool\_id (PK), scheme\_id (FK, UNIQUE), financial\_year, total\_budget, disbursed\_amount
- Officer — officer\_id (PK), officer\_name, employee\_code, department\_id (FK), designation, assigned\_district, assigned\_state
- Citizen — citizen\_id (PK), aadhaar\_number (UNIQUE), full\_name, gender, date\_of\_birth, age, category, annual\_income, occupation, land\_holding\_acres, location\_type, district, state, bank\_account (UNIQUE), is\_verified
- Citizen\_Documents — doc\_id (PK), citizen\_id (FK), doc\_type, doc\_number, status, verified\_by (FK → Officer)
- Application — application\_id (PK), citizen\_id (FK), scheme\_id (FK), officer\_id (FK), apply\_date, status, priority\_score, approval\_date
- Fund\_Disbursement — disbursement\_id (PK), application\_id (FK, UNIQUE), amount, payment\_mode, transaction\_ref, bank\_account
- Application\_Audit\_Log — audit\_id (PK), application\_id (FK), old\_status, new\_status, changed\_by, change\_date

Key relationships:

- Department — Scheme: One-to-Many (one department owns many schemes)
- Scheme — Scheme\_Eligibility\_Rules: One-to-One (each scheme has one rule set)
- Scheme — Scheme\_Fund\_Pool: One-to-One (each scheme has one fund pool)
- Citizen — Application: One-to-Many (one citizen may apply to multiple schemes)
- Scheme — Application: One-to-Many (one scheme receives many applications)
- Application — Fund\_Disbursement: One-to-One (one approved application yields one disbursement)
- Application — Application\_Audit\_Log: One-to-Many (every status change is logged)

## 4. ER TO TABLES

### 1. DEPARTMENT

| Attribute        | Data Type     | Constraint(s)          |
|------------------|---------------|------------------------|
| department_id    | NUMBER        | PRIMARY KEY (IDENTITY) |
| department_name  | VARCHAR2(100) | NOT NULL, UNIQUE       |
| department_code  | VARCHAR2(10)  | NOT NULL, UNIQUE       |
| ministry         | VARCHAR2(100) | NOT NULL               |
| head_name        | VARCHAR2(100) |                        |
| established_year | NUMBER(4)     | CHECK (1947–2025)      |

## 2. SCHEME

| Attribute           | Data Type     | Constraint(s)                |
|---------------------|---------------|------------------------------|
| scheme_id           | NUMBER        | PRIMARY KEY (IDENTITY)       |
| scheme_name         | VARCHAR2(150) | NOT NULL, UNIQUE             |
| scheme_code         | VARCHAR2(20)  | NOT NULL, UNIQUE             |
| department_id       | NUMBER        | FK → Department              |
| scheme_type         | VARCHAR2(10)  | CHECK ('MAJOR','MINOR')      |
| description         | VARCHAR2(500) |                              |
| launch_date         | DATE          | NOT NULL                     |
| base_benefit_amount | NUMBER(12,2)  | CHECK (> 0)                  |
| max_benefit_amount  | NUMBER(12,2)  | CHECK (≥ base)               |
| is_active           | CHAR(1)       | DEFAULT 'Y', CHECK ('Y','N') |

## 3. SCHEME\_ELIGIBILITY\_RULES

| Attribute          | Data Type    | Constraint(s)                 |
|--------------------|--------------|-------------------------------|
| rule_id            | NUMBER       | PRIMARY KEY (IDENTITY)        |
| scheme_id          | NUMBER       | FK → Scheme                   |
| min_age            | NUMBER(3)    | DEFAULT 18                    |
| max_age            | NUMBER(3)    | DEFAULT 120                   |
| max_income         | NUMBER(12,2) |                               |
| min_income         | NUMBER(12,2) | DEFAULT 0                     |
| allowed_categories | VARCHAR2(50) | DEFAULT 'ALL'                 |
| location_type      | VARCHAR2(10) | CHECK ('RURAL','URBAN','ALL') |
| gender_restriction | VARCHAR2(5)  | CHECK ('M','F','ALL')         |
| min_land_acres     | NUMBER(6,2)  | DEFAULT 0                     |

## 4. SCHEME\_FUND\_POOL

| Attribute      | Data Type    | Constraint(s)          |
|----------------|--------------|------------------------|
| pool_id        | NUMBER       | PRIMARY KEY (IDENTITY) |
| scheme_id      | NUMBER       | FK → Scheme, UNIQUE    |
| financial_year | VARCHAR2(10) | NOT NULL               |
| total_budget   | NUMBER(15,2) | NOT NULL, CHECK (> 0)  |

|                  |              |                 |
|------------------|--------------|-----------------|
| disbursed_amount | NUMBER(15,2) | DEFAULT 0       |
| last_updated     | DATE         | DEFAULT SYSDATE |

## 5. OFFICER

| Attribute         | Data Type     | Constraint(s)          |
|-------------------|---------------|------------------------|
| officer_id        | NUMBER        | PRIMARY KEY (IDENTITY) |
| officer_name      | VARCHAR2(100) | NOT NULL               |
| employee_code     | VARCHAR2(20)  | NOT NULL, UNIQUE       |
| department_id     | NUMBER        | FK → Department        |
| designation       | VARCHAR2(100) |                        |
| assigned_district | VARCHAR2(100) | NOT NULL               |
| assigned_state    | VARCHAR2(100) | NOT NULL               |
| join_date         | DATE          | NOT NULL               |
| is_active         | CHAR(1)       | DEFAULT 'Y'            |

## 6. CITIZEN

| Attribute          | Data Type     | Constraint(s)                  |
|--------------------|---------------|--------------------------------|
| citizen_id         | NUMBER        | PRIMARY KEY (IDENTITY)         |
| aadhaar_number     | CHAR(12)      | NOT NULL, UNIQUE, REGEX \d{12} |
| full_name          | VARCHAR2(100) | NOT NULL                       |
| gender             | CHAR(1)       | CHECK ('M','F')                |
| date_of_birth      | DATE          | NOT NULL                       |
| age                | NUMBER(3)     | CHECK (0–120)                  |
| category           | VARCHAR2(5)   | CHECK ('SC','ST','OBC','GEN')  |
| annual_income      | NUMBER(12,2)  | NOT NULL, CHECK (≥ 0)          |
| occupation         | VARCHAR2(100) |                                |
| land_holding_acres | NUMBER(6,2)   | DEFAULT 0                      |
| location_type      | VARCHAR2(10)  | CHECK ('RURAL','URBAN')        |
| district           | VARCHAR2(100) | NOT NULL                       |
| state              | VARCHAR2(100) | NOT NULL                       |
| bank_account       | VARCHAR2(20)  | NOT NULL, UNIQUE               |
| ifsc_code          | VARCHAR2(11)  | NOT NULL                       |
| is_verified        | CHAR(1)       | DEFAULT 'N'                    |

## 7. CITIZEN\_DOCUMENTS

| Attribute   | Data Type    | Constraint(s)                           |
|-------------|--------------|-----------------------------------------|
| doc_id      | NUMBER       | PRIMARY KEY (IDENTITY)                  |
| citizen_id  | NUMBER       | FK → Citizen                            |
| doc_type    | VARCHAR2(50) | CHECK (valid doc types)                 |
| doc_number  | VARCHAR2(50) |                                         |
| upload_date | DATE         | DEFAULT SYSDATE                         |
| status      | VARCHAR2(15) | CHECK ('PENDING','VERIFIED','REJECTED') |
| verified_by | NUMBER       | FK → Officer                            |

## 8. APPLICATION

| Attribute        | Data Type     | Constraint(s)                  |
|------------------|---------------|--------------------------------|
| application_id   | NUMBER        | PRIMARY KEY (IDENTITY)         |
| citizen_id       | NUMBER        | FK → Citizen                   |
| scheme_id        | NUMBER        | FK → Scheme                    |
| officer_id       | NUMBER        | FK → Officer                   |
| apply_date       | DATE          | NOT NULL, DEFAULT SYSDATE      |
| status           | VARCHAR2(20)  | CHECK (valid statuses)         |
| priority_score   | NUMBER(5,2)   | DEFAULT 0                      |
| rejection_reason | VARCHAR2(300) |                                |
| approval_date    | DATE          |                                |
|                  |               | UNIQUE (citizen_id, scheme_id) |

## 9. FUND\_DISBURSEMENT

| Attribute         | Data Type    | Constraint(s)                |
|-------------------|--------------|------------------------------|
| disbursement_id   | NUMBER       | PRIMARY KEY (IDENTITY)       |
| application_id    | NUMBER       | FK → Application, UNIQUE     |
| amount            | NUMBER(12,2) | NOT NULL, CHECK (> 0)        |
| disbursement_date | DATE         | DEFAULT SYSDATE              |
| payment_mode      | VARCHAR2(10) | CHECK ('DBT','NEFT','RTGS')  |
| transaction_ref   | VARCHAR2(50) | UNIQUE                       |
| bank_account      | VARCHAR2(20) | NOT NULL                     |
| status            | VARCHAR2(15) | CHECK ('PROCESSED','FAILED') |

## 10. APPLICATION\_AUDIT\_LOG

| Attribute      | Data Type     | Constraint(s)          |
|----------------|---------------|------------------------|
| audit_id       | NUMBER        | PRIMARY KEY (IDENTITY) |
| application_id | NUMBER        | FK → Application       |
| old_status     | VARCHAR2(20)  |                        |
| new_status     | VARCHAR2(20)  | NOT NULL               |
| changed_by     | VARCHAR2(100) | DEFAULT USER           |
| change_date    | TIMESTAMP     | DEFAULT SYSTIMESTAMP   |

## 5. NORMALISED TABLES

All tables have been normalised to Third Normal Form (3NF). The list below shows the final schema with primary keys underlined (represented in bold) and foreign keys noted.

- DEPARTMENT (department\_id, department\_name, department\_code, ministry, head\_name, established\_year)
- SCHEME (scheme\_id, scheme\_name, scheme\_code, department\_id [FK], scheme\_type, description, launch\_date, base\_benefit\_amount, max\_benefit\_amount, applicable\_states, is\_active)
- SCHEME\_ELIGIBILITY\_RULES (rule\_id, scheme\_id [FK], min\_age, max\_age, max\_income, min\_income, allowed\_categories, location\_type, gender\_restriction, min\_land\_acres)
- SCHEME\_FUND\_POOL (pool\_id, scheme\_id [FK, UNIQUE], financial\_year, total\_budget, disbursed\_amount, last\_updated)
- OFFICER (officer\_id, officer\_name, employee\_code, department\_id [FK], designation, assigned\_district, assigned\_state, phone, join\_date, is\_active)
- CITIZEN (citizen\_id, aadhaar\_number, full\_name, gender, date\_of\_birth, age, category, annual\_income, occupation, land\_holding\_acres, location\_type, village\_town, district, state, pincode, phone, bank\_account, ifsc\_code, is\_verified, registration\_date)
- CITIZEN\_DOCUMENTS (doc\_id, citizen\_id [FK], doc\_type, doc\_number, upload\_date, status, verified\_by [FK])
- APPLICATION (application\_id, citizen\_id [FK], scheme\_id [FK], officer\_id [FK], apply\_date, status, priority\_score, rejection\_reason, approval\_date, remarks)
- FUND\_DISBURSEMENT (disbursement\_id, application\_id [FK, UNIQUE], amount, disbursement\_date, payment\_mode, transaction\_ref, bank\_account, status)
- APPLICATION\_AUDIT\_LOG (audit\_id, application\_id [FK], old\_status, new\_status, changed\_by, change\_date)

2NF compliance: All non-key attributes in each table depend on the full primary key, not a partial key. For example, all citizen attributes depend entirely on citizen\_id.

3NF compliance: No transitive dependencies exist. Scheme details are in the Scheme table and not repeated in Application. Officer details are not duplicated in Audit\_Log.

## 6. SQL CODE — TABLE DEFINITIONS & SAMPLE DATA

### CREATE TABLE Statements

#### Department

|                           |                                                          |
|---------------------------|----------------------------------------------------------|
| CREATE TABLE Department ( |                                                          |
| department_id             | NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,         |
| department_name           | VARCHAR2(100) NOT NULL UNIQUE,                           |
| department_code           | VARCHAR2(10) NOT NULL UNIQUE,                            |
| ministry                  | VARCHAR2(100) NOT NULL,                                  |
| head_name                 | VARCHAR2(100),                                           |
| established_year          | NUMBER(4) CHECK (established_year BETWEEN 1947 AND 2025) |
| );                        |                                                          |

#### Scheme

|                                                                                             |                                                        |
|---------------------------------------------------------------------------------------------|--------------------------------------------------------|
| CREATE TABLE Scheme (                                                                       |                                                        |
| scheme_id                                                                                   | NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,       |
| scheme_name                                                                                 | VARCHAR2(150) NOT NULL UNIQUE,                         |
| scheme_code                                                                                 | VARCHAR2(20) NOT NULL UNIQUE,                          |
| department_id                                                                               | NUMBER NOT NULL,                                       |
| scheme_type                                                                                 | VARCHAR2(10) CHECK (scheme_type IN ('MAJOR','MINOR')), |
| description                                                                                 | VARCHAR2(500),                                         |
| launch_date                                                                                 | DATE NOT NULL,                                         |
| base_benefit_amount                                                                         | NUMBER(12,2) CHECK (base_benefit_amount > 0),          |
| max_benefit_amount                                                                          | NUMBER(12,2) CHECK (max_benefit_amount > 0),           |
| applicable_states                                                                           | VARCHAR2(500) DEFAULT 'ALL',                           |
| is_active                                                                                   | CHAR(1) DEFAULT 'Y' CHECK (is_active IN ('Y','N')),    |
| CONSTRAINT fk_scheme_dept FOREIGN KEY (department_id) REFERENCES Department(department_id), |                                                        |
| CONSTRAINT chk_benefit CHECK (max_benefit_amount >= base_benefit_amount)                    |                                                        |
| );                                                                                          |                                                        |

#### Scheme\_Eligibility\_Rules

|                                         |                                                  |
|-----------------------------------------|--------------------------------------------------|
| CREATE TABLE Scheme_Eligibility_Rules ( |                                                  |
| rule_id                                 | NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, |
| scheme_id                               | NUMBER NOT NULL,                                 |
| min_age                                 | NUMBER(3) DEFAULT 18,                            |
| max_age                                 | NUMBER(3) DEFAULT 120,                           |
| max_income                              | NUMBER(12,2),                                    |
| min_income                              | NUMBER(12,2) DEFAULT 0,                          |
| allowed_categories                      | VARCHAR2(50) DEFAULT 'ALL',                      |
| location_type                           | VARCHAR2(10) DEFAULT 'ALL'                       |

|                                                                                |
|--------------------------------------------------------------------------------|
| CHECK (location_type IN ('RURAL','URBAN','ALL')),                              |
| gender_restriction VARCHAR2(5) DEFAULT 'ALL'                                   |
| CHECK (gender_restriction IN ('M','F','ALL')),                                 |
| min_land_acres NUMBER(6,2) DEFAULT 0,                                          |
| CONSTRAINT fk_rule_scheme FOREIGN KEY (scheme_id) REFERENCES Scheme(scheme_id) |
| );                                                                             |

## Application

|                                                                                    |
|------------------------------------------------------------------------------------|
| CREATE TABLE Application (                                                         |
| application_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,                    |
| citizen_id NUMBER NOT NULL,                                                        |
| scheme_id NUMBER NOT NULL,                                                         |
| officer_id NUMBER,                                                                 |
| apply_date DATE DEFAULT SYSDATE NOT NULL,                                          |
| status VARCHAR2(20) DEFAULT 'SUBMITTED'                                            |
| CHECK (status IN ('SUBMITTED','UNDER_REVIEW',                                      |
| 'APPROVED','REJECTED','DISBURSED')),                                               |
| priority_score NUMBER(5,2) DEFAULT 0,                                              |
| rejection_reason VARCHAR2(300),                                                    |
| approval_date DATE,                                                                |
| remarks VARCHAR2(400),                                                             |
| CONSTRAINT fk_app_citizen FOREIGN KEY (citizen_id) REFERENCES Citizen(citizen_id), |
| CONSTRAINT fk_app_scheme FOREIGN KEY (scheme_id) REFERENCES Scheme(scheme_id),     |
| CONSTRAINT fk_app_officer FOREIGN KEY (officer_id) REFERENCES Officer(officer_id), |
| CONSTRAINT uq_citizen_scheme UNIQUE (citizen_id, scheme_id)                        |
| );                                                                                 |

## Fund\_Disbursement

|                                                                  |
|------------------------------------------------------------------|
| CREATE TABLE Fund_Disbursement (                                 |
| disbursement_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY, |
| application_id NUMBER NOT NULL UNIQUE,                           |
| amount NUMBER(12,2) NOT NULL CHECK (amount > 0),                 |
| disbursement_date DATE DEFAULT SYSDATE,                          |
| payment_mode VARCHAR2(10) DEFAULT 'DBT'                          |
| CHECK (payment_mode IN ('DBT','NEFT','RTGS')),                   |
| transaction_ref VARCHAR2(50) UNIQUE,                             |
| bank_account VARCHAR2(20) NOT NULL,                              |
| status VARCHAR2(15) DEFAULT 'PROCESSED'                          |
| CHECK (status IN ('PROCESSED','FAILED')),                        |

|                                                     |
|-----------------------------------------------------|
| CONSTRAINT fk_disb_app FOREIGN KEY (application_id) |
| REFERENCES Application(application_id)              |
| );                                                  |

## Sample Data

The system is pre-loaded with 7 departments, 10 schemes (5 major, 5 minor), 10 eligibility rule sets, 10 fund pools for financial year 2024-25, and 30 officers across Indian states. Below is an extract of the scheme inserts:

|                                                                           |
|---------------------------------------------------------------------------|
| -- Major Scheme: Kisan Samman Yojana                                      |
| INSERT INTO Scheme (scheme_name, scheme_code, department_id, scheme_type, |
| description, launch_date, base_benefit_amount, max_benefit_amount)        |
| VALUES ('Kisan Samman Yojana', 'KSY-2019', 1, 'MAJOR',                    |
| 'Annual income support for small farmers. Paid in three instalments via   |
| DBT.',                                                                    |
| DATE '2019-02-24', 6000, 8000);                                           |
| -- Minor Scheme: SC ST Scholarship Scheme                                 |
| INSERT INTO Scheme (scheme_name, scheme_code, department_id, scheme_type, |
| description, launch_date, base_benefit_amount, max_benefit_amount)        |
| VALUES ('SC ST Scholarship Scheme', 'SCSS-2008', 6, 'MINOR',              |
| 'Merit-cum-means scholarship for SC/ST students in higher education.',    |
| DATE '2008-07-01', 12000, 36000);                                         |

## 7. SQL CODE — QUERIES (30 QUERIES)

### Section 1 — Basic Validation Queries

#### Q1. List All Schemes With Their Department

Displays all welfare schemes along with their parent department and benefit ranges, grouped by type.

|                                                         |
|---------------------------------------------------------|
| SELECT s.scheme_name, s.scheme_type, d.department_name, |
| s.base_benefit_amount, s.is_active                      |
| FROM Scheme s                                           |
| JOIN Department d ON s.department_id = d.department_id  |
| ORDER BY s.scheme_type, s.scheme_name;                  |

#### Q2. All Citizens From a Specific State

Retrieves all citizens registered from Uttar Pradesh, sorted by income to identify the most economically vulnerable.

|                                              |
|----------------------------------------------|
| SELECT full_name, state, district, category, |
| annual_income, location_type                 |

|                               |
|-------------------------------|
| FROM Citizen                  |
| WHERE state = 'Uttar Pradesh' |
| ORDER BY annual_income ASC;   |

### Q3. Applications With Citizen and Scheme Details

Joins three tables to produce a full application register with citizen identification and scheme name.

|                                                         |
|---------------------------------------------------------|
| SELECT a.application_id, c.full_name, c.aadhaar_number, |
| s.scheme_name, a.status, a.apply_date, a.priority_score |
| FROM Application a                                      |
| JOIN Citizen c ON a.citizen_id = c.citizen_id           |
| JOIN Scheme s ON a.scheme_id = s.scheme_id              |
| ORDER BY a.apply_date DESC;                             |

### Q4. All Officers With Department and Assignment

Lists the full officer roster sorted by assigned state, useful for verification of coverage across districts.

|                                                          |
|----------------------------------------------------------|
| SELECT o.officer_name, o.designation, d.department_name, |
| o.assigned_district, o.assigned_state                    |
| FROM Officer o                                           |
| JOIN Department d ON o.department_id = d.department_id   |
| ORDER BY o.assigned_state;                               |

### Q5. All Active Schemes With Budget Summary

Shows only active schemes with their budget allocation and remaining balance from the fund pool.

|                                                             |
|-------------------------------------------------------------|
| SELECT s.scheme_name, s.scheme_type,                        |
| sfp.total_budget, sfp.disbursed_amount,                     |
| sfp.total_budget - sfp.disbursed_amount AS remaining_budget |
| FROM Scheme s                                               |
| JOIN Scheme_Fund_Pool sfp ON s.scheme_id = sfp.scheme_id    |
| WHERE s.is_active = 'Y'                                     |
| ORDER BY sfp.total_budget DESC;                             |

### Q6. Documents Pending Verification

Lists all citizen documents awaiting officer verification, sorted by upload date to process older documents first.

|                                                  |
|--------------------------------------------------|
| SELECT cd.doc_id, c.full_name, c.aadhaar_number, |
| cd.doc_type, cd.upload_date, cd.status           |
| FROM Citizen_Documents cd                        |

```

JOIN Citizen c ON cd.citizen_id = c.citizen_id
WHERE cd.status = 'PENDING'
ORDER BY cd.upload_date ASC;

```

## Q7. Eligibility Rules for All Schemes

Displays the complete eligibility criteria for each scheme from the rule engine table.

```

SELECT s.scheme_name, r.min_age, r.max_age,
       r.max_income, r.allowed_categories,
       r.location_type, r.gender_restriction, r.min_land_acres
FROM Scheme_Eligibility_Rules r
JOIN Scheme s ON r.scheme_id = s.scheme_id
ORDER BY s.scheme_name;

```

## Section 2 — Business Logic Queries

### Q8. Application Count Per Scheme Broken by Status

Provides a status breakdown per scheme — useful for tracking the processing pipeline and identifying bottlenecks.

```

SELECT s.scheme_name, a.status,
       COUNT(*) AS count
FROM Application a
JOIN Scheme s ON a.scheme_id = s.scheme_id
GROUP BY s.scheme_name, a.status
ORDER BY s.scheme_name, a.status;

```

### Q9. Citizens Eligible for Swasthya Suraksha Who Have Not Applied

Identifies verified citizens who qualify for the Swasthya Suraksha Yojana health scheme but have not yet applied, supporting outreach campaigns.

```

SELECT c.citizen_id, c.full_name, c.state,
       c.category, c.annual_income
FROM Citizen c
WHERE c.annual_income < 500000
      AND c.is_verified = 'Y'
      AND NOT EXISTS (
        SELECT 1 FROM Application a
        WHERE a.citizen_id = c.citizen_id AND a.scheme_id = 3
      )
ORDER BY c.annual_income ASC;

```

### Q10. Officer Daily Queue — Pending Applications by Priority

Builds an officer's work queue, sorting pending applications by priority score (highest vulnerability first) and then by date of submission.

|                                                   |
|---------------------------------------------------|
| SELECT a.application_id, c.full_name, c.category, |
| c.annual_income, s.scheme_name,                   |
| a.priority_score, a.apply_date,                   |
| TRUNC(SYSDATE - a.apply_date) AS days_waiting     |
| FROM Application a                                |
| JOIN Citizen c ON a.citizen_id = c.citizen_id     |
| JOIN Scheme s ON a.scheme_id = s.scheme_id        |
| WHERE a.status IN ('SUBMITTED','UNDER_REVIEW')    |
| ORDER BY a.priority_score DESC, a.apply_date ASC; |

### Q11. Schemes With More Than 50 Applications

Identifies popular schemes that have received significant application volume, which may require additional officer allocation.

|                                                      |
|------------------------------------------------------|
| SELECT s.scheme_name,                                |
| COUNT(a.application_id) AS total_applications        |
| FROM Scheme s                                        |
| LEFT JOIN Application a ON s.scheme_id = a.scheme_id |
| GROUP BY s.scheme_name                               |
| HAVING COUNT(a.application_id) > 50                  |
| ORDER BY total_applications DESC;                    |

### Q12. Citizens With Multiple Scheme Applications

Finds citizens who have applied to more than one scheme, helping track multi-scheme beneficiaries for policy analysis.

|                                                                 |
|-----------------------------------------------------------------|
| SELECT c.citizen_id, c.full_name, c.category,                   |
| c.annual_income,                                                |
| COUNT(a.application_id) AS total_applications                   |
| FROM Citizen c                                                  |
| JOIN Application a ON c.citizen_id = a.citizen_id               |
| GROUP BY c.citizen_id, c.full_name, c.category, c.annual_income |
| HAVING COUNT(a.application_id) > 1                              |
| ORDER BY total_applications DESC;                               |

### Q13. Disbursements Made Within the Last 30 Days

Retrieves all successful DBT disbursements made in the past 30 days — useful for month-end reconciliation.

|                                                            |
|------------------------------------------------------------|
| SELECT fd.disbursement_id, c.full_name, s.scheme_name,     |
| fd.amount, fd.disbursement_date,                           |
| fd.payment_mode, fd.transaction_ref                        |
| FROM Fund_Disbursement fd                                  |
| JOIN Application a ON fd.application_id = a.application_id |
| JOIN Citizen c ON a.citizen_id = c.citizen_id              |
| JOIN Scheme s ON a.scheme_id = s.scheme_id                 |
| WHERE fd.disbursement_date >= SYSDATE - 30                 |
| AND fd.status = 'PROCESSED'                                |
| ORDER BY fd.disbursement_date DESC;                        |

### Q14. Rejected Applications With Rejection Reasons

Lists all rejected applications with the reason recorded by the officer, enabling grievance analysis and appeals tracking.

|                                                         |
|---------------------------------------------------------|
| SELECT a.application_id, c.full_name, c.aadhaar_number, |
| s.scheme_name, a.rejection_reason, a.approval_date      |
| FROM Application a                                      |
| JOIN Citizen c ON a.citizen_id = c.citizen_id           |
| JOIN Scheme s ON a.scheme_id = s.scheme_id              |
| WHERE a.status = 'REJECTED'                             |
| ORDER BY a.approval_date DESC;                          |

## Section 3 — Analytics & Insights

### Q15. Total Funds Disbursed Per Scheme

Provides a financial summary per scheme showing total funds disbursed, count of beneficiaries paid, and average benefit amount.

|                                                                        |
|------------------------------------------------------------------------|
| SELECT s.scheme_name,                                                  |
| NVL(SUM(fd.amount), 0) AS total_disbursed,                             |
| COUNT(fd.disbursement_id) AS disbursement_count,                       |
| ROUND(NVL(AVG(fd.amount), 0), 2) AS avg_per_beneficiary                |
| FROM Scheme s                                                          |
| LEFT JOIN Application a ON s.scheme_id = a.scheme_id                   |
| LEFT JOIN Fund_Disbursement fd ON a.application_id = fd.application_id |
| GROUP BY s.scheme_name                                                 |
| ORDER BY total_disbursed DESC;                                         |

### Q16. Scheme Budget Utilisation With Health Status

Calculates budget utilisation percentage for each scheme and classifies it as HEALTHY, MODERATE, or CRITICAL — enabling finance officers to take timely action.

|                                                                        |
|------------------------------------------------------------------------|
| SELECT s.scheme_name,                                                  |
| sfp.total_budget, sfp.disbursed_amount,                                |
| sfp.total_budget - sfp.disbursed_amount AS remaining,                  |
| ROUND((sfp.disbursed_amount / sfp.total_budget) * 100, 2) AS used_pct, |
| CASE                                                                   |
| WHEN sfp.disbursed_amount / sfp.total_budget >= 0.9 THEN 'CRITICAL'    |
| WHEN sfp.disbursed_amount / sfp.total_budget >= 0.5 THEN 'MODERATE'    |
| ELSE 'HEALTHY'                                                         |
| END AS budget_status                                                   |
| FROM Scheme s                                                          |
| JOIN Scheme_Fund_Pool sfp ON s.scheme_id = sfp.scheme_id               |
| ORDER BY used_pct DESC;                                                |

### Q17. Rural vs Urban Beneficiary Breakdown Per Scheme

Breaks down approved beneficiaries and disbursed funds by rural/urban residence for each scheme, supporting geographic equity analysis.

|                                                                        |
|------------------------------------------------------------------------|
| SELECT s.scheme_name, c.location_type,                                 |
| COUNT(DISTINCT a.citizen_id) AS beneficiaries,                         |
| NVL(SUM(fd.amount), 0) AS total_amount                                 |
| FROM Scheme s                                                          |
| JOIN Application a ON s.scheme_id = a.scheme_id                        |
| JOIN Citizen c ON a.citizen_id = c.citizen_id                          |
| LEFT JOIN Fund_Disbursement fd ON a.application_id = fd.application_id |
| WHERE a.status IN ('APPROVED','DISBURSED')                             |
| GROUP BY s.scheme_name, c.location_type                                |
| ORDER BY s.scheme_name, c.location_type;                               |

### Q18. State-wise Disbursed Applications and Total Funds

Shows state-wise fund distribution, helping policymakers identify which states are receiving the most welfare support and which are under-served.

|                                                     |
|-----------------------------------------------------|
| SELECT c.state,                                     |
| COUNT(DISTINCT a.application_id) AS disbursed_apps, |
| NVL(SUM(fd.amount), 0) AS total_funds,              |
| ROUND(AVG(fd.amount), 0) AS avg_benefit             |
| FROM Citizen c                                      |

|                                                                        |
|------------------------------------------------------------------------|
| JOIN Application a ON c.citizen_id = a.citizen_id                      |
| LEFT JOIN Fund_Disbursement fd ON a.application_id = fd.application_id |
| WHERE a.status IN ('APPROVED','DISBURSED')                             |
| GROUP BY c.state                                                       |
| ORDER BY total_funds DESC;                                             |

### Q19. Category-wise Breakdown of Benefits Received

Analyses scheme benefit distribution across SC, ST, OBC, and GEN categories to verify social equity objectives are being met.

|                                                                        |
|------------------------------------------------------------------------|
| SELECT c.category,                                                     |
| COUNT(DISTINCT c.citizen_id) AS total_citizens,                        |
| COUNT(a.application_id) AS applications,                               |
| SUM(CASE WHEN a.status IN ('APPROVED','DISBURSED')                     |
| THEN 1 ELSE 0 END) AS approved,                                        |
| NVL(SUM(fd.amount), 0) AS total_benefit                                |
| FROM Citizen c                                                         |
| LEFT JOIN Application a ON c.citizen_id = a.citizen_id                 |
| LEFT JOIN Fund_Disbursement fd ON a.application_id = fd.application_id |
| GROUP BY c.category                                                    |
| ORDER BY total_benefit DESC;                                           |

### Q20. Scheme-wise Average Priority Score of Approved Applicants

Checks whether higher-priority (more vulnerable) citizens are being approved — a fairness quality metric for the scheme.

|                                                        |
|--------------------------------------------------------|
| SELECT s.scheme_name,                                  |
| ROUND(AVG(a.priority_score), 2) AS avg_priority_score, |
| MIN(a.priority_score) AS min_score,                    |
| MAX(a.priority_score) AS max_score                     |
| FROM Application a                                     |
| JOIN Scheme s ON a.scheme_id = s.scheme_id             |
| WHERE a.status IN ('APPROVED','DISBURSED')             |
| GROUP BY s.scheme_name                                 |
| ORDER BY avg_priority_score DESC;                      |

### Q21. Monthly Application Submission Trend

Tracks monthly application volumes and disbursement counts to identify seasonal trends or processing surges.

|                                                 |
|-------------------------------------------------|
| SELECT TO_CHAR(apply_date, 'YYYY-MM') AS month, |
| COUNT(*) AS total_applications,                 |

|                                                                    |
|--------------------------------------------------------------------|
| SUM(CASE WHEN status = 'DISBURSED' THEN 1 ELSE 0 END) AS disbursed |
| FROM Application                                                   |
| GROUP BY TO_CHAR(apply_date, 'YYYY-MM')                            |
| ORDER BY month;                                                    |

## Section 4 — Advanced Queries

### Q22. Citizens Below State Average Income Not Yet Applied

Uses a correlated subquery to find verified citizens earning below their state's average income who have never applied — primary targets for proactive outreach.

|                                                               |
|---------------------------------------------------------------|
| SELECT c.citizen_id, c.full_name, c.state,                    |
| c.annual_income, c.category                                   |
| FROM Citizen c                                                |
| WHERE c.annual_income < (                                     |
| SELECT AVG(annual_income) FROM Citizen c2                     |
| WHERE c2.state = c.state                                      |
| )                                                             |
| AND NOT EXISTS (                                              |
| SELECT 1 FROM Application a WHERE a.citizen_id = c.citizen_id |
| )                                                             |
| AND c.is_verified = 'Y'                                       |
| ORDER BY c.annual_income ASC;                                 |

### Q23. Fraud Detection — High Income Citizens on BPL Schemes

Detects potential fraudulent applications where citizens with incomes above Rs. 5 lakh have applied for BPL-targeted schemes like Kisan Samman, Gramin Awas, or Ujjwala.

|                                                        |
|--------------------------------------------------------|
| SELECT c.full_name, c.aadhaar_number, c.annual_income, |
| c.category, s.scheme_name, a.status, a.apply_date      |
| FROM Application a                                     |
| JOIN Citizen c ON a.citizen_id = c.citizen_id          |
| JOIN Scheme s ON a.scheme_id = s.scheme_id             |
| WHERE c.annual_income > 500000                         |
| AND s.scheme_id IN (1, 2, 5)                           |
| ORDER BY c.annual_income DESC;                         |

### Q24. Officer Approval Rate and Average Processing Time

Measures officer performance by computing approval rate percentage and average processing time in days for each active officer.

|                                          |
|------------------------------------------|
| SELECT o.officer_name, o.assigned_state, |
|------------------------------------------|

|                                                                                    |
|------------------------------------------------------------------------------------|
| COUNT(a.application_id) AS handled,                                                |
| SUM(CASE WHEN a.status IN ('APPROVED','DISBURSED') THEN 1 ELSE 0 END) AS approved, |
| SUM(CASE WHEN a.status = 'REJECTED' THEN 1 ELSE 0 END) AS rejected,                |
| ROUND(                                                                             |
| SUM(CASE WHEN a.status IN ('APPROVED','DISBURSED') THEN 1 ELSE 0 END)              |
| * 100.0                                                                            |
| / NULLIF(COUNT(a.application_id), 0), 1                                            |
| ) AS approval_rate_pct,                                                            |
| ROUND(AVG(CASE WHEN a.approval_date IS NOT NULL                                    |
| THEN a.approval_date - a.apply_date END), 1) AS avg_days_taken                     |
| FROM Officer o                                                                     |
| LEFT JOIN Application a ON o.officer_id = a.officer_id                             |
| GROUP BY o.officer_name, o.assigned_state                                          |
| HAVING COUNT(a.application_id) > 0                                                 |
| ORDER BY approval_rate_pct DESC;                                                   |

## Q25. Top 5 Schemes by Total Amount Disbursed

Ranks the top five schemes by cumulative disbursement amount — indicating which programmes have the highest financial reach.

|                                                                        |
|------------------------------------------------------------------------|
| SELECT s.scheme_name,                                                  |
| NVL(SUM(fd.amount), 0) AS total_disbursed                              |
| FROM Scheme s                                                          |
| LEFT JOIN Application a ON s.scheme_id = a.scheme_id                   |
| LEFT JOIN Fund_Disbursement fd ON a.application_id = fd.application_id |
| GROUP BY s.scheme_name                                                 |
| ORDER BY total_disbursed DESC                                          |
| FETCH FIRST 5 ROWS ONLY;                                               |

## Q26. Unverified Citizens With Pending Applications

Finds citizens whose documents have not been verified yet but who have active pending applications — a compliance risk requiring immediate officer attention.

|                                                          |
|----------------------------------------------------------|
| SELECT c.citizen_id, c.full_name, c.aadhaar_number,      |
| c.is_verified, a.application_id, s.scheme_name, a.status |
| FROM Citizen c                                           |
| JOIN Application a ON c.citizen_id = a.citizen_id        |
| JOIN Scheme s ON a.scheme_id = s.scheme_id               |
| WHERE c.is_verified = 'N'                                |
| AND a.status IN ('SUBMITTED','UNDER_REVIEW')             |
| ORDER BY a.apply_date ASC;                               |

### Q27. Scheme Applications With No Assigned Officer

Identifies pending applications not yet assigned to any officer, with the number of days they have been waiting — for workload allocation.

|                                                  |
|--------------------------------------------------|
| SELECT a.application_id, c.full_name,            |
| s.scheme_name, a.status, a.apply_date,           |
| TRUNC(SYSDATE - a.apply_date) AS days_unassigned |
| FROM Application a                               |
| JOIN Citizen c ON a.citizen_id = c.citizen_id    |
| JOIN Scheme s ON a.scheme_id = s.scheme_id       |
| WHERE a.officer_id IS NULL                       |
| AND a.status IN ('SUBMITTED','UNDER_REVIEW')     |
| ORDER BY days_unassigned DESC;                   |

### Q28. Audit Trail for a Specific Application

Retrieves the complete audit trail for a specific application, showing every status transition, who made it, and when — for accountability and dispute resolution.

|                                        |
|----------------------------------------|
| SELECT al.audit_id, al.application_id, |
| al.old_status, al.new_status,          |
| al.changed_by, al.change_date          |
| FROM Application_Audit_Log al          |
| WHERE al.application_id = 1            |
| ORDER BY al.change_date ASC;           |

### Q29. Department-wise Total Budget and Disbursement

Rolls up financial data at the department (ministry) level, showing how much of each ministry's total allocation has been spent across all its schemes.

|                                                           |
|-----------------------------------------------------------|
| SELECT d.department_name,                                 |
| COUNT(s.scheme_id) AS total_schemes,                      |
| SUM(sfp.total_budget) AS total_budget,                    |
| SUM(sfp.disbursed_amount) AS total_disbursed,             |
| ROUND(SUM(sfp.disbursed_amount) * 100.0                   |
| / NULLIF(SUM(sfp.total_budget), 0), 2) AS utilization_pct |
| FROM Department d                                         |
| JOIN Scheme s ON d.department_id = s.department_id        |
| JOIN Scheme_Fund_Pool sfp ON s.scheme_id = sfp.scheme_id  |
| GROUP BY d.department_name                                |
| ORDER BY total_disbursed DESC;                            |

### Q30. Full Application Pipeline View — Latest Status Per Citizen

Provides a comprehensive end-to-end view of every application — from submission through to disbursement — combining citizen profile, scheme details, and payment information in one result set.

|                                                                        |
|------------------------------------------------------------------------|
| SELECT c.full_name, c.state, c.category,                               |
| s.scheme_name, a.status,                                               |
| a.priority_score, a.apply_date, a.approval_date,                       |
| fd.amount AS disbursed_amount,                                         |
| fd.transaction_ref                                                     |
| FROM Application a                                                     |
| JOIN Citizen c ON a.citizen_id = c.citizen_id                          |
| JOIN Scheme s ON a.scheme_id = s.scheme_id                             |
| LEFT JOIN Fund_Disbursement fd ON a.application_id = fd.application_id |
| ORDER BY a.priority_score DESC, a.apply_date ASC;                      |

## 8. PL/SQL CODE

### 8.1 Functions

#### FUNCTION 1: check\_eligibility

Accepts a citizen\_id and scheme\_id, reads the Scheme\_Eligibility\_Rules table, and returns 'ELIGIBLE' or a descriptive reason string (e.g., 'INELIGIBLE: Age 75 out of range [18-70]'). This function drives the rule engine — no eligibility logic is hard-coded elsewhere.

|                                                                           |
|---------------------------------------------------------------------------|
| CREATE OR REPLACE FUNCTION check_eligibility(                             |
| p_citizen_id IN NUMBER,                                                   |
| p_scheme_id IN NUMBER                                                     |
| ) RETURN VARCHAR2 AS                                                      |
| v_age NUMBER; v_income NUMBER; v_category VARCHAR2(5);                    |
| v_location VARCHAR2(10); v_gender CHAR(1); v_land NUMBER;                 |
| v_active CHAR(1);                                                         |
| v_min_age NUMBER; v_max_age NUMBER; v_max_inc NUMBER;                     |
| v_min_inc NUMBER; v_cats VARCHAR2(50);                                    |
| v_loc_req VARCHAR2(10); v_gen_req VARCHAR2(5); v_min_land NUMBER;         |
| BEGIN                                                                     |
| SELECT is_active INTO v_active FROM Scheme WHERE scheme_id = p_scheme_id; |
| IF v_active = 'N' THEN                                                    |
| RETURN 'INELIGIBLE: Scheme is currently inactive';                        |
| END IF;                                                                   |
| SELECT age, annual_income, category, location_type, gender,               |
| land_holding_acres                                                        |
| INTO v_age, v_income, v_category, v_location, v_gender, v_land            |
| FROM Citizen WHERE citizen_id = p_citizen_id;                             |
| SELECT min_age, max_age, max_income, min_income,                          |

```

        allowed_categories, location_type, gender_restriction, min_land_acres
    INTO v_min_age, v_max_age, v_max_inc, v_min_inc,
        v_cats, v_loc_req, v_gen_req, v_min_land
    FROM Scheme_Eligibility_Rules WHERE scheme_id = p_scheme_id;
    IF v_age < v_min_age OR v_age > v_max_age THEN
        RETURN 'INELIGIBLE: Age out of range';
    END IF;
    IF v_max_inc IS NOT NULL AND v_income > v_max_inc THEN
        RETURN 'INELIGIBLE: Income exceeds limit';
    END IF;
    -- ... (additional checks for category, location, gender, land)
    RETURN 'ELIGIBLE';
EXCEPTION
    WHEN NO_DATA_FOUND THEN RETURN 'INELIGIBLE: Data not found';
END check_eligibility;
/

```

## FUNCTION 2: calculate\_benefit

Computes the actual benefit amount for a citizen-scheme pair by applying income-tier scaling (factor 0.40 to 1.00), then category bonuses (+20% for SC/ST, +10% for OBC), a rural location bonus (+8%), and an elderly bonus (+12% for age ≥ 60). The result is capped at max\_benefit\_amount.

```

CREATE OR REPLACE FUNCTION calculate_benefit(
    p_citizen_id IN NUMBER,
    p_scheme_id  IN NUMBER
) RETURN NUMBER AS
    v_base NUMBER; v_max NUMBER; v_income NUMBER;
    v_category VARCHAR2(5); v_location VARCHAR2(10);
    v_age NUMBER; v_benefit NUMBER; v_factor NUMBER;
BEGIN
    SELECT base_benefit_amount, max_benefit_amount
    INTO v_base, v_max
    FROM Scheme WHERE scheme_id = p_scheme_id;

    SELECT annual_income, category, location_type, age
    INTO v_income, v_category, v_location, v_age
    FROM Citizen WHERE citizen_id = p_citizen_id;

    IF    v_income <= 50000  THEN v_factor := 1.0;
    ELSIF v_income <= 150000 THEN v_factor := 0.88;
    ELSIF v_income <= 300000 THEN v_factor := 0.75;
    ELSIF v_income <= 500000 THEN v_factor := 0.60;
    ELSE
        v_factor := 0.40; END IF;

```

|                                                                          |
|--------------------------------------------------------------------------|
|                                                                          |
| v_benefit := v_base * v_factor;                                          |
| IF v_category IN ('SC','ST') THEN v_benefit := v_benefit * 1.20; END IF; |
| IF v_location = 'RURAL' THEN v_benefit := v_benefit * 1.08; END IF;      |
| IF v_age >= 60 THEN v_benefit := v_benefit * 1.12; END IF;               |
| IF v_benefit > v_max THEN v_benefit := v_max; END IF;                    |
| RETURN ROUND(v_benefit, 2);                                              |
| EXCEPTION WHEN OTHERS THEN RETURN 0;                                     |
| END calculate_benefit;                                                   |
| /                                                                        |

### FUNCTION 3: get\_priority\_score

Returns a score out of 100 used to rank applications in the officer queue. Income accounts for up to 50 points (inverse relationship), social category for 25 points, rural location for 15 points, and age above 60 for 10 points.

|                                                             |
|-------------------------------------------------------------|
| CREATE OR REPLACE FUNCTION get_priority_score(              |
| p_citizen_id IN NUMBER                                      |
| ) RETURN NUMBER AS                                          |
| v_income NUMBER; v_category VARCHAR2(5);                    |
| v_location VARCHAR2(10); v_age NUMBER; v_score NUMBER := 0; |
| BEGIN                                                       |
| SELECT annual_income, category, location_type, age          |
| INTO v_income, v_category, v_location, v_age                |
| FROM Citizen WHERE citizen_id = p_citizen_id;               |
|                                                             |
| -- Income: up to 50 points (lower income = higher score)    |
| IF v_income <= 50000 THEN v_score := v_score + 50;          |
| ELSIF v_income <= 100000 THEN v_score := v_score + 44;      |
| ELSIF v_income <= 200000 THEN v_score := v_score + 36;      |
| ELSIF v_income <= 350000 THEN v_score := v_score + 26;      |
| ELSIF v_income <= 500000 THEN v_score := v_score + 16;      |
| ELSE v_score := v_score + 5; END IF;                        |
|                                                             |
| -- Category: ST=25, SC=22, OBC=14, GEN=5                    |
| IF v_category = 'ST' THEN v_score := v_score + 25;          |
| ELSIF v_category = 'SC' THEN v_score := v_score + 22;       |
| ELSIF v_category = 'OBC' THEN v_score := v_score + 14;      |
| ELSE v_score := v_score + 5; END IF;                        |
|                                                             |
| -- Rural: +15, Urban: +5                                    |
| IF v_location = 'RURAL' THEN v_score := v_score + 15;       |
| ELSE v_score := v_score + 5; END IF;                        |

|                                                      |
|------------------------------------------------------|
|                                                      |
| -- Age 60+: bonus 10                                 |
| IF v_age >= 60 THEN v_score := v_score + 10; END IF; |
|                                                      |
| RETURN v_score;                                      |
| EXCEPTION WHEN OTHERS THEN RETURN 0;                 |
| END get_priority_score;                              |
| /                                                    |

#### FUNCTION 4: is\_duplicate\_app

Returns 'Y' if the citizen already has a non-REJECTED application for the given scheme, 'N' otherwise. Used as a guard in the apply\_scheme procedure before inserting a new application.

|                                                     |
|-----------------------------------------------------|
| CREATE OR REPLACE FUNCTION is_duplicate_app(        |
| p_citizen_id IN NUMBER,                             |
| p_scheme_id  IN NUMBER                              |
| ) RETURN CHAR AS                                    |
| v_count NUMBER;                                     |
| BEGIN                                               |
| SELECT COUNT(*) INTO v_count                        |
| FROM Application                                    |
| WHERE citizen_id = p_citizen_id                     |
| AND scheme_id = p_scheme_id                         |
| AND status NOT IN ('REJECTED');                     |
| RETURN CASE WHEN v_count > 0 THEN 'Y' ELSE 'N' END; |
| EXCEPTION WHEN OTHERS THEN RETURN 'N';              |
| END is_duplicate_app;                               |
| /                                                   |

#### FUNCTION 5: get\_scheme\_utilization

Returns the percentage of a scheme's budget that has been disbursed. Used in dashboards and budget monitoring views.

|                                                      |
|------------------------------------------------------|
| CREATE OR REPLACE FUNCTION get_scheme_utilization(   |
| p_scheme_id IN NUMBER                                |
| ) RETURN NUMBER AS                                   |
| v_total NUMBER; v_spent NUMBER;                      |
| BEGIN                                                |
| SELECT total_budget, disbursed_amount                |
| INTO v_total, v_spent                                |
| FROM Scheme_Fund_Pool WHERE scheme_id = p_scheme_id; |
| IF v_total = 0 THEN RETURN 0; END IF;                |

```

        RETURN ROUND((v_spent / v_total) * 100, 2);
    EXCEPTION WHEN NO_DATA_FOUND THEN RETURN 0;
END get_scheme_utilization;
/

```

## 8.2 Stored Procedures

### PROCEDURE 1: generate\_citizen\_data

Generates 1,000 realistic citizen records using DBMS\_RANDOM and hand-curated name arrays. State distribution is population-weighted (Uttar Pradesh has the highest probability). Income follows a right-skewed distribution matching Indian census data: 40% of citizens earn below Rs. 1 lakh.

```

CREATE OR REPLACE PROCEDURE generate_citizen_data AS
    TYPE str_arr IS TABLE OF VARCHAR2(60);
    v_male_names    str_arr := str_arr('Rajesh','Suresh','Arjun',...);
    v_female_names  str_arr := str_arr('Sunita','Kavita','Fatima',...);
    v_surnames      str_arr := str_arr('Sharma','Singh','Patel',...);
    v_states        str_arr := str_arr('Uttar Pradesh',... ); -- weighted
    -- ... (other arrays for districts, categories, occupations)
BEGIN
    FOR i IN 1..1000 LOOP
        -- gender, name, age, state, income, category selection
        -- using DBMS_RANDOM.VALUE()
        INSERT INTO Citizen (...) VALUES (...);
        IF MOD(i, 100) = 0 THEN COMMIT; END IF;
    END LOOP;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Done. 1000 citizens generated. ');
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN ROLLBACK;
    WHEN OTHERS THEN ROLLBACK;
END generate_citizen_data;
/
BEGIN generate_citizen_data; END;
/

```

### PROCEDURE 2: generate\_sample\_applications

Uses three explicit cursors to create realistic applications: c\_farmers fetches up to 200 farmer-type citizens for Kisan Samman Yojana, c\_bpl\_rural fetches 150 rural BPL citizens for Gramin Awas Yojana, and c\_health fetches 250 low-income citizens for Swasthya Suraksha. Status is randomly drawn from a weighted distribution replicating real pipeline proportions.

```

CREATE OR REPLACE PROCEDURE generate_sample_applications AS
    CURSOR c_farmers IS

```

```

SELECT citizen_id, annual_income, category, location_type
FROM Citizen
WHERE occupation IN ('Farmer','Agricultural Labourer')
AND land_holding_acres BETWEEN 0.1 AND 5
AND annual_income < 250000
AND ROWNUM <= 200;

TYPE status_list IS TABLE OF VARCHAR2(20);
v_statuses status_list := status_list(
'SUBMITTED','SUBMITTED','UNDER_REVIEW',
'APPROVED','APPROVED','REJECTED','DISBURSED');
BEGIN
FOR r IN c_farmers LOOP
BEGIN
INSERT INTO Application (...) VALUES (...);
EXCEPTION WHEN DUP_VAL_ON_INDEX THEN NULL;
END;
END LOOP;
-- similar loops for c_bpl_rural and c_health
COMMIT;
DBMS_OUTPUT.PUT_LINE('Sample applications generated.');
```

END generate\_sample\_applications;

/

### PROCEDURE 3: apply\_scheme

The main public-facing procedure for submitting an application. It validates citizen and scheme existence, checks for duplicate non-rejected applications, evaluates eligibility, computes the priority score, and inserts the application with status SUBMITTED.

```

CREATE OR REPLACE PROCEDURE apply_scheme(
p_citizen_id IN NUMBER,
p_scheme_id IN NUMBER,
p_remarks IN VARCHAR2 DEFAULT NULL
) AS
v_eligible VARCHAR2(200); v_duplicate CHAR(1);
v_score NUMBER; v_cname VARCHAR2(100); v_sname VARCHAR2(150);
BEGIN
-- existence check: citizen
SELECT full_name INTO v_cname FROM Citizen
WHERE citizen_id = p_citizen_id;

-- existence check: active scheme
SELECT scheme_name INTO v_sname
```

|                                                                    |
|--------------------------------------------------------------------|
| FROM Scheme WHERE scheme_id = p_scheme_id AND is_active = 'Y';     |
|                                                                    |
| -- duplicate guard                                                 |
| IF is_duplicate_app(p_citizen_id, p_scheme_id) = 'Y' THEN          |
| RAISE_APPLICATION_ERROR(-20003, 'Already has active application'); |
| END IF;                                                            |
|                                                                    |
| v_eligible := check_eligibility(p_citizen_id, p_scheme_id);        |
| v_score := get_priority_score(p_citizen_id);                       |
|                                                                    |
| INSERT INTO Application (citizen_id, scheme_id, apply_date,        |
| status, priority_score, remarks)                                   |
| VALUES (p_citizen_id, p_scheme_id, SYSDATE,                        |
| 'SUBMITTED', v_score, NVL(p_remarks, 'Self-applied'));             |
| COMMIT;                                                            |
| DBMS_OUTPUT.PUT_LINE('Application submitted. Score: '    v_score); |
| DBMS_OUTPUT.PUT_LINE('Eligibility: '    v_eligible);               |
| EXCEPTION                                                          |
| WHEN OTHERS THEN ROLLBACK; RAISE;                                  |
| END apply_scheme;                                                  |
| /                                                                  |

#### PROCEDURE 4: approve\_application

The officer-facing approval workflow. Locks the application row, validates it is still pending, checks fund availability, calls calculate\_benefit(), inserts a Fund\_Disbursement record, updates the Scheme\_Fund\_Pool balance, and advances the status to DISBURSED — all atomically. Rejection records the reason and sets status to REJECTED.

|                                                              |
|--------------------------------------------------------------|
| CREATE OR REPLACE PROCEDURE approve_application(             |
| p_app_id        IN NUMBER,                                   |
| p_officer_id IN NUMBER,                                      |
| p_action        IN VARCHAR2,    -- 'APPROVE' or 'REJECT'     |
| p_remarks      IN VARCHAR2 DEFAULT NULL                      |
| ) AS                                                         |
| v_citizen_id NUMBER; v_scheme_id NUMBER;                     |
| v_curr_status VARCHAR2(20); v_benefit NUMBER;                |
| v_bank VARCHAR2(20); v_txn VARCHAR2(50); v_fund_left NUMBER; |
| BEGIN                                                        |
| SELECT citizen_id, scheme_id, status                         |
| INTO v_citizen_id, v_scheme_id, v_curr_status                |
| FROM Application WHERE application_id = p_app_id FOR UPDATE; |
|                                                              |
| IF v_curr_status NOT IN ('SUBMITTED', 'UNDER_REVIEW') THEN   |

```

        RAISE_APPLICATION_ERROR(-20010, 'Already processed');
    END IF;

    IF UPPER(p_action) = 'APPROVE' THEN
        SELECT total_budget - disbursed_amount INTO v_fund_left
        FROM Scheme_Fund_Pool WHERE scheme_id = v_scheme_id;
        v_benefit := calculate_benefit(v_citizen_id, v_scheme_id);
        IF v_fund_left < v_benefit THEN
            RAISE_APPLICATION_ERROR(-20011, 'Insufficient funds');
        END IF;
        -- approve + create disbursement + update pool
        UPDATE Application SET status='DISBURSED', approval_date=SYSDATE
        WHERE application_id = p_app_id;
        COMMIT;
    ELSIF UPPER(p_action) = 'REJECT' THEN
        UPDATE Application SET status='REJECTED',
            rejection_reason = NVL(p_remarks, 'Rejected during review')
        WHERE application_id = p_app_id;
        COMMIT;
    END IF;

    EXCEPTION WHEN OTHERS THEN ROLLBACK; RAISE;
END approve_application;
/

```

## PROCEDURE 5: generate\_scheme\_report

A cursor-based reporting procedure that prints a formatted scheme performance report to DBMS\_OUTPUT. It shows scheme name, type, disbursement in crores, total applications, and pending count, with a grand total footer.

```

CREATE OR REPLACE PROCEDURE generate_scheme_report AS
    CURSOR c_report IS
        SELECT s.scheme_name, s.scheme_type,
            sfp.total_budget, sfp.disbursed_amount,
            COUNT(a.application_id) AS total_apps,
            SUM(CASE WHEN a.status='DISBURSED' THEN 1 ELSE 0 END) AS done,
            SUM(CASE WHEN a.status IN ('SUBMITTED', 'UNDER_REVIEW')
                THEN 1 ELSE 0 END) AS pending
        FROM Scheme s
        JOIN Scheme_Fund_Pool sfp ON s.scheme_id = sfp.scheme_id
        LEFT JOIN Application a ON s.scheme_id = a.scheme_id
        GROUP BY s.scheme_name, s.scheme_type,
            sfp.total_budget, sfp.disbursed_amount
        ORDER BY sfp.disbursed_amount DESC;

```

|                                                            |
|------------------------------------------------------------|
| r c_report%ROWTYPE;                                        |
| BEGIN                                                      |
| DBMS_OUTPUT.PUT_LINE('=== SCHEME PERFORMANCE REPORT ==='); |
| OPEN c_report;                                             |
| LOOP                                                       |
| FETCH c_report INTO r;                                     |
| EXIT WHEN c_report%NOTFOUND;                               |
| DBMS_OUTPUT.PUT_LINE(r.scheme_name    '   '                |
| r.total_apps    ' apps   Pending: '    r.pending);         |
| END LOOP;                                                  |
| CLOSE c_report;                                            |
| EXCEPTION                                                  |
| WHEN OTHERS THEN                                           |
| IF c_report%ISOPEN THEN CLOSE c_report; END IF;            |
| END generate_scheme_report;                                |
| /                                                          |
| BEGIN generate_scheme_report; END;                         |
| /                                                          |

## 8.3 Triggers

### TRIGGER 1: trg\_application\_audit

An AFTER UPDATE trigger on the Application table that fires whenever the status column changes. It inserts a record into Application\_Audit\_Log capturing the old status, new status, the Oracle user who made the change, and a precise timestamp. This provides a tamper-proof audit trail.

|                                                                 |
|-----------------------------------------------------------------|
| CREATE OR REPLACE TRIGGER trg_application_audit                 |
| AFTER UPDATE OF status ON Application                           |
| FOR EACH ROW                                                    |
| BEGIN                                                           |
| INSERT INTO Application_Audit_Log (                             |
| application_id, old_status, new_status, changed_by, change_date |
| ) VALUES (                                                      |
| :NEW.application_id, :OLD.status, :NEW.status,                  |
| USER, SYSTIMESTAMP                                              |
| );                                                              |
| END trg_application_audit;                                      |
| /                                                               |

Sample output after approving application 1:

|                                                               |                |            |            |            |             |
|---------------------------------------------------------------|----------------|------------|------------|------------|-------------|
| SELECT * FROM Application_Audit_Log WHERE application_id = 1; |                |            |            |            |             |
|                                                               |                |            |            |            |             |
| AUDIT ID                                                      | APPLICATION_ID | OLD STATUS | NEW STATUS | CHANGED BY | CHANGE DATE |

|                 |   |           |           |       |           |
|-----------------|---|-----------|-----------|-------|-----------|
| -----           |   |           |           |       |           |
| -----           |   |           |           |       |           |
| 1               | 1 | SUBMITTED | APPROVED  | ADMIN | 03-MAY-26 |
| 10.32.15.000000 |   |           |           |       |           |
| 2               | 1 | APPROVED  | DISBURSED | ADMIN | 03-MAY-26 |
| 10.32.15.012345 |   |           |           |       |           |

## TRIGGER 2: trg\_fund\_pool\_update

An AFTER INSERT trigger on Fund\_Disbursement that automatically keeps the Scheme\_Fund\_Pool balance accurate. When a disbursement record is inserted, it increments disbursed\_amount and updates last\_updated on the corresponding fund pool row. This ensures the pool is always consistent without requiring manual updates.

|                                                              |
|--------------------------------------------------------------|
| CREATE OR REPLACE TRIGGER trg_fund_pool_update               |
| AFTER INSERT ON Fund_Disbursement                            |
| FOR EACH ROW                                                 |
| DECLARE                                                      |
| v_scheme_id NUMBER;                                          |
| BEGIN                                                        |
| SELECT scheme_id INTO v_scheme_id                            |
| FROM Application WHERE application_id = :NEW.application_id; |
|                                                              |
| UPDATE Scheme_Fund_Pool                                      |
| SET disbursed_amount = disbursed_amount + :NEW.amount,       |
| last_updated = SYSDATE                                       |
| WHERE scheme_id = v_scheme_id;                               |
| EXCEPTION                                                    |
| WHEN NO_DATA_FOUND THEN                                      |
| DBMS_OUTPUT.PUT_LINE('Warning: scheme not found');           |
| END trg_fund_pool_update;                                    |
| /                                                            |

## TRIGGER 3: trg\_no\_duplicate\_app

A BEFORE INSERT trigger on Application that prevents duplicate applications. Before any row is inserted, it checks whether the citizen already has a non-REJECTED application for the same scheme. If a duplicate is found, it raises an application error with a descriptive message — providing a database-level safeguard in addition to the procedure-level check.

|                                                |
|------------------------------------------------|
| CREATE OR REPLACE TRIGGER trg_no_duplicate_app |
| BEFORE INSERT ON Application                   |
| FOR EACH ROW                                   |
| DECLARE                                        |
| v_count NUMBER; v_sname VARCHAR2(150);         |
| BEGIN                                          |

```

SELECT COUNT(*) INTO v_count
FROM Application
WHERE citizen_id = :NEW.citizen_id
AND scheme_id = :NEW.scheme_id
AND status NOT IN ('REJECTED');

IF v_count > 0 THEN
    SELECT scheme_name INTO v_sname
    FROM Scheme WHERE scheme_id = :NEW.scheme_id;
    RAISE_APPLICATION_ERROR(-20030,
        'Active application already exists for: ' || v_sname);
END IF;
END trg_no_duplicate_app;
/

```

Test — attempting to insert a duplicate application:

```

-- First application succeeds
INSERT INTO Application (citizen_id, scheme_id, apply_date, status)
VALUES (1, 1, SYSDATE, 'SUBMITTED');

-- Second application for same citizen + scheme triggers error:
INSERT INTO Application (citizen_id, scheme_id, apply_date, status)
VALUES (1, 1, SYSDATE, 'SUBMITTED');

OUTPUT:
ORA-20030: Active application already exists for: Kisan Samman Yojana
ORA-06512: at "GSMS.TRG_NO_DUPLICATE_APP", line 10

```

## 9. CONCLUSION

This Government Scheme Management System demonstrates how a well-designed relational database can serve as the backbone of a complex welfare administration platform. The system handles the complete lifecycle of a government scheme — from department setup and budget allocation through citizen registration, eligibility checking, application submission, officer review, fund disbursement, and post-disbursement audit logging — entirely within Oracle SQL and PL/SQL.

Key design achievements of this project include:

- A table-driven eligibility rule engine that makes adding new scheme criteria a data operation, not a code change.
- A dynamic benefit calculation function that ensures more vulnerable citizens (lower income, SC/ST, rural, elderly) receive higher benefit amounts within the scheme's cap.
- A priority scoring system that allows officers to process the most needy applications first, reducing subjective bias.
- Atomic transaction management in stored procedures — fund pool updates, disbursement inserts, and status transitions are rolled back together if any step fails.

- Automatic audit logging via triggers, providing a complete and tamper-proof history of every application status change.
- Comprehensive SQL queries across 30 use cases covering basic validation, business logic, analytics, and fraud detection.

Through this project, the concepts of ER design, schema normalisation, PL/SQL programming, trigger-based automation, and query optimisation have been applied to a real-world problem that directly affects millions of Indian citizens. The system can serve as the SQL layer for a full-stack government portal, requiring only a frontend and API layer to be production-ready.

## 10. REFERENCES

1. Oracle Live SQL Platform — <https://livesql.oracle.com>
2. Oracle PL/SQL Documentation — <https://docs.oracle.com/en/database/oracle/oracle-database/21/lnpls/>
3. MyFitnessPal (inspiration for tracking apps) — <https://www.myfitnesspal.com/>
4. Government of India — PM Kisan Samman Nidhi — <https://pmkisan.gov.in/>
5. Government of India — Pradhan Mantri Awas Yojana — <https://pmaymis.gov.in/>
6. Government of India — Ayushman Bharat — <https://pmjay.gov.in/>
7. Ministry of Rural Development — MGNREGS — <https://nrega.nic.in/>
8. India Census 2011 — Population and Category Data — <https://censusindia.gov.in/>
9. DBMS\_RANDOM Oracle Documentation — [https://docs.oracle.com/cd/B19306\\_01/appdev.102/b14258/d\\_random.htm](https://docs.oracle.com/cd/B19306_01/appdev.102/b14258/d_random.htm)